scraper_runner

June 1, 2026

[ ]:
```
# ================== SCRAPERUNNER - CELDA 1/7: IMPORTS + PARÁMETROS GLOBALES ==================
# (ANTES: CELDA 1/6) -> (AHORA: CELDA 1/7) [sin cambios funcionales, solo renumeración]

import os  # Manejo de archivos y directorios del sistema operativo
import re  # Expresiones regulares para búsqueda y manipulación de texto
import time  # Funciones relacionadas con tiempo (pausas, medir duración)
import random  # Generación de números aleatorios
from typing import Optional, Tuple, List  # Tipado estático para mayor claridad

import numpy as np  # Operaciones matemáticas y manejo de arreglos
import pandas as pd  # Manejo de datos en tablas (DataFrames)
from bs4 import BeautifulSoup  # Parsing y extracción de información de HTML/XML

from selenium import webdriver  # Control del navegador
from selenium.webdriver.chrome.service import Service  # Configuración del servicio de ChromeDriver
from selenium.webdriver.chrome.options import Options  # Opciones de configuración del navegador
from webdriver_manager.chrome import ChromeDriverManager  # Descarga y gestión automática de ChromeDriver

from selenium.webdriver.common.by import By  # Localización de elementos en la página
from selenium.webdriver.common.action_chains import ActionChains  # Simulación de acciones complejas (clics, arrastrar)
from selenium.webdriver.common.keys import Keys  # Simulación de teclas del teclado (ej. CTRL+CLICK)
from selenium.webdriver.support.ui import WebDriverWait  # Esperas explícitas hasta que se cumpla una condición
from selenium.webdriver.support import expected_conditions as EC  # Condiciones para las esperas (ej. que aparezca un elemento)

from selenium.common.exceptions import TimeoutException  # Manejo de errores cuando una espera excede el tiempo límite
```

```
PAGINA_INICIO = 1
PAGINA_FIN = 40
BASE_URL = "https://inmuebles.mercadolibre.com.mx/casas/venta/distrito-federal/
 ↪_DisplayType_M"

OBJETIVO_LIMPIOS = 5000
OBJETIVO_CRUDO_MINIMO = 6000
TIPO_CAMBIO_USD_MXN = 18.0

RAW_BASENAME = "Casas_raw"
LIMPIO_BASENAME = "Casas_limpio"
RAW_CSV = f"{RAW_BASENAME}.csv"
LIMPIO_CSV = f"{LIMPIO_BASENAME}.csv"

WAIT_TIMEOUT = 20
PAGELOAD_TIMEOUT = 60

# ===== PERFIL DEDICADO (RECOMENDADO) =====
CHROME_USER_DATA_DIR = r"C:\chrome_profiles\ml"  # carpeta nueva dedicada
CHROME_PROFILE_DIR = "Default"                   # déjalo así
```

[ ]:
```
# ================== SCRAPERUNNER - CELDA 2/7: UTILIDADES (TIMER + PARSEO +␣
 ↪EXTRACTORES) ==================
# (ANTES: CELDA 2/6) -> (AHORA: CELDA 2/7) [sin cambios funcionales, solo␣
 ↪renumeración]

class HumanTimer:
    def __init__(self):
        self.last_delay = random.uniform(0.1, 0.8)

    def wait(self, min_s=0.2, max_s=1.0, extra_jitter=(0.02, 0.5)):
        if min_s < 0:
            min_s = 0.0
        if max_s < min_s:
            max_s = min_s + 0.1

        base = random.uniform(min_s, max_s)
        correlated = 0.7 * base + 0.3 * self.last_delay
        jitter = random.uniform(*extra_jitter)
        if random.random() < 0.3:
            jitter *= random.uniform(1.0, 2.5)

        delay = max(0.05, correlated + jitter)
        if random.random() < 0.1:
            delay += random.uniform(0.05, 0.4)

        self.last_delay = delay
```

```
        time.sleep(delay)


def parsear_precio_mxn(texto: Optional[str]) -> float:
    if texto is None or (isinstance(texto, float) and np.isnan(texto)):
        return np.nan

    s = str(texto)
    s_lower = s.lower()

    moneda = "MXN"
    if ("usd" in s_lower or "us$" in s_lower or "u$s" in s_lower or
        "dólar" in s_lower or "dolar" in s_lower):
        moneda = "USD"

    m = re.search(r"(\d[\d,\.]*)", s)
    if not m:
        return np.nan

    valor_str = m.group(1).replace(",", "")
    try:
        valor = float(valor_str)
    except ValueError:
        return np.nan

    if moneda == "USD":
        valor *= TIPO_CAMBIO_USD_MXN

    return valor


def extraer_recamaras_superficie(card) -> Tuple[Optional[float],␣
 ↪Optional[float]]:
    txt = card.get_text(" ", strip=True).lower()

    recamaras = None
    superficie = None

    m_rec = re.search(r"(\d+)\s*rec[aá]m", txt)
    if m_rec:
        recamaras = float(m_rec.group(1))

    m_sup = re.search(r"(\d+(?:\.\d+)?)\s*(m2|m²)", txt.replace(",", ""))
    if m_sup:
        superficie = float(m_sup.group(1))

    return recamaras, superficie
```

```
def extraer_banos_desde_texto(texto: Optional[str]) -> Optional[float]:
    if not texto:
        return None
    txt = str(texto).lower()
    m = re.search(r"(\d+(?:\.\d+)?)\s*bañ", txt)
    if not m:
        return None
    try:
        return float(m.group(1))
    except ValueError:
        return None
```

[ ]:
```
# ================== SCRAPERUNNER - CELDA 4/7: SCROLL + CLICK SIGUIENTE (FIX␣
 ↪ROBUSTO v2.30.x) ==================
# Qué cambia vs antes:
# - click_siguiente ahora es multi-estrategia (selectors + XPath + JS click␣
 ↪fallback)
# - hace scroll al fondo antes de buscar el botón
# - reintenta 3 veces y valida disabled
# - NO cambia nada del resto del scraping (solo paginación)

def human_scroll_page(driver: webdriver.Chrome, timer: HumanTimer, min_steps=3,␣
 ↪max_steps=6):
    try:
        total_height = driver.execute_script("return document.body.
 ↪scrollHeight") or 0
    except Exception:
        return

    if total_height <= 0:
        return

    steps = random.randint(min_steps, max_steps)
    for _ in range(steps):
        target = int(total_height * random.uniform(0.1, 0.95))
        driver.execute_script("window.scrollTo(0, arguments[0]);", target)
        timer.wait(0.2, 0.9, extra_jitter=(0.0, 0.3))

    if random.random() < 0.4:
        target = int(total_height * random.uniform(0.0, 0.4))
        driver.execute_script("window.scrollTo(0, arguments[0]);", target)
        timer.wait(0.3, 1.0, extra_jitter=(0.0, 0.3))


def click_siguiente(driver: webdriver.Chrome, timer: HumanTimer) -> bool:
```

```
    """
    Click robusto en 'Siguiente' para MercadoLibre (andes-pagination).
    Retorna True si avanzó, False si no hay más o no pudo.
    """
    for _ in range(3):
        try:
            # Scroll al final: el paginador vive abajo
            try:
                driver.execute_script("window.scrollTo(0, document.body.
↪scrollHeight);")
            except Exception:
                pass
            timer.wait(0.7, 1.8, extra_jitter=(0.05, 0.4))

            # ===== Estrategia 1: selector estable de andes-pagination =====
            candidatos = []
            try:
                candidatos = driver.find_elements(
                    By.CSS_SELECTOR,
                    "a.andes-pagination__link--next, button.
↪andes-pagination__link--next"
                )
            except Exception:
                candidatos = []

            # ===== Estrategia 2: fallback por texto "Siguiente" =====
            if not candidatos:
                try:
                    candidatos = driver.find_elements(
                        By.XPATH,
                        "//a[contains(normalize-space(.), 'Siguiente')] | //
↪button[contains(normalize-space(.), 'Siguiente')]"
                    )
                except Exception:
                    candidatos = []

            if not candidatos:
                return False

            next_btn = candidatos[0]

            # Validar deshabilitado
            aria_disabled = (next_btn.get_attribute("aria-disabled") or "").
↪lower()
            disabled_attr = (next_btn.get_attribute("disabled") or "")
            classes = (next_btn.get_attribute("class") or "").lower()
```

```
            if "true" in aria_disabled or disabled_attr or "disabled" in␣
 ↪classes:
                return False

            # Intento click normal
            try:
                ActionChains(driver)\
                    .move_to_element(next_btn)\
                    .pause(random.uniform(0.1, 0.4))\
                    .click()\
                    .perform()
            except Exception:
                # Fallback JS click (útil si hay overlay/intercept)
                try:
                    driver.execute_script("arguments[0].click();", next_btn)
                except Exception:
                    timer.wait(0.6, 1.2, extra_jitter=(0.05, 0.35))
                    continue

            timer.wait(2.0, 5.0, extra_jitter=(0.1, 0.7))
            return True

        except Exception:
            timer.wait(0.6, 1.3, extra_jitter=(0.05, 0.35))
            continue

    return False
```

[ ]:
```
# ================== SCRAPERUNNER - CELDA 4/7: SCROLL + CLICK SIGUIENTE␣
 ↪==================
# (ANTES: CELDA 4/6) -> (AHORA: CELDA 4/7) [sin cambios funcionales, solo␣
 ↪renumeración]

def human_scroll_page(driver: webdriver.Chrome, timer: HumanTimer, min_steps=3,␣
 ↪max_steps=6):
    try:
        total_height = driver.execute_script("return document.body.
 ↪scrollHeight") or 0
    except Exception:
        return

    if total_height <= 0:
        return

    steps = random.randint(min_steps, max_steps)
    for _ in range(steps):
        target = int(total_height * random.uniform(0.1, 0.95))
```

```
        driver.execute_script("window.scrollTo(0, arguments[0]);", target)
        timer.wait(0.2, 0.9, extra_jitter=(0.0, 0.3))

    if random.random() < 0.4:
        target = int(total_height * random.uniform(0.0, 0.4))
        driver.execute_script("window.scrollTo(0, arguments[0]);", target)
        timer.wait(0.3, 1.0, extra_jitter=(0.0, 0.3))


def click_siguiente(driver: webdriver.Chrome, timer: HumanTimer) -> bool:
    try:
        human_scroll_page(driver, timer, min_steps=2, max_steps=4)
        timer.wait(0.8, 2.0, extra_jitter=(0.1, 0.5))

        xpath = (
            "//span[contains(@class,'andes-pagination__arrow-title') "
            "and normalize-space()='Siguiente']"
        )
        span_next = WebDriverWait(driver, WAIT_TIMEOUT).until(
            EC.presence_of_element_located((By.XPATH, xpath))
        )
        next_btn = span_next.find_element(By.XPATH, "./ancestor::a | ./ancestor:
 ↪:button")

        aria_disabled = (next_btn.get_attribute("aria-disabled") or "").lower()
        classes = (next_btn.get_attribute("class") or "").lower()
        if "true" in aria_disabled or "disabled" in classes:
            return False

        ActionChains(driver).move_to_element(next_btn).pause(random.uniform(0.
 ↪2, 0.8)).click().perform()
        timer.wait(2.0, 5.0, extra_jitter=(0.1, 0.7))
        return True

    except Exception:
        return False
```

[ ]:
```
# ================== SCRAPERUNNER - CELDA 5/7 (RESTORE estable)
# - Ubicacion (texto crudo)
# - Maps_url (link real Google Maps)
# - Lat, Lng (coordenadas del center)
# - NUNCA guarda staticmap en Maps_url
# [IMPORTANTE]: NO METE POI aquí para no romper maps/lat/lng

import re
from urllib.parse import urlparse, parse_qs, unquote
from typing import Optional, Tuple
```

```
def _contar_unidades_en_detalle_html(html: str) -> int:
    soup = BeautifulSoup(html, "html.parser")
    units = soup.select("div.ui-vip-available-units__unit-container")
    return len(units) if units else 1


def _parse_staticmap(src: str) -> Tuple[Optional[str], Optional[float],␣
 ↪Optional[float], Optional[int]]:
    """
    Del src de staticmap extrae:
      - maps_url (google maps navegable)
      - lat (float)
      - lng (float)
      - zoom (int) (opcional)
    """
    if not src:
        return None, None, None, None

    src = src.replace("&amp;", "&").strip()
    if "maps.googleapis.com/maps/api/staticmap" not in src:
        return None, None, None, None

    try:
        parsed = urlparse(src)
        qs = parse_qs(parsed.query)

        center = (qs.get("center", [None])[0] or "").strip()
        zoom_s = (qs.get("zoom", [None])[0] or "").strip()

        if not center:
            return None, None, None, None

        center = unquote(center).replace("%2C", ",").replace("%2c", ",")
        m = re.match(r"^\s*([-\d\.]+)\s*,\s*([-\d\.]+)\s*$", center)
        if not m:
            return None, None, None, None

        lat_s, lng_s = m.group(1), m.group(2)
        lat = float(lat_s)
        lng = float(lng_s)

        zoom = None
        if zoom_s:
            try:
                zoom = int(float(zoom_s))
            except Exception:
```

```
                zoom = None

        z = str(zoom) if zoom is not None else "16"
        maps_url = f"https://www.google.com/maps?q={lat},{lng}&z={z}"

        return maps_url, lat, lng, zoom

    except Exception:
        return None, None, None, None


def _extraer_ubicacion_y_maps_html(html: str) -> Tuple[Optional[str],␣
↪Optional[str], Optional[float], Optional[float]]:
    soup = BeautifulSoup(html, "html.parser")

    # ===== UBICACION (texto crudo) =====
    ubicacion = None
    loc_span = soup.select_one("div.ui-vip-location p.ui-pdp-media__title >␣
↪span")
    if loc_span:
        ubicacion = loc_span.get_text(" ", strip=True) or None
    if not ubicacion:
        loc_p = soup.select_one("div.ui-vip-location p.ui-pdp-media__title")
        if loc_p:
            ubicacion = loc_p.get_text(" ", strip=True) or None

    # ===== MAPS_URL + LAT/LNG =====
    maps_url = None
    lat = None
    lng = None

    img_map = soup.select_one("#ui-vip-location__map img")
    if img_map:
        src = img_map.get("src") or ""
        maps_url, lat, lng, _zoom = _parse_staticmap(src)

    return ubicacion, maps_url, lat, lng


def _abrir_anuncio_en_nueva_pestana(driver: webdriver.Chrome, timer:␣
↪HumanTimer, card_el) -> bool:
    handles_before = driver.window_handles[:]
    try:
        a = card_el.find_element(By.CSS_SELECTOR, "a[href]")
    except Exception:
        a = card_el
```

```
    try:
        ActionChains(driver)\
            .move_to_element(a)\
            .pause(random.uniform(0.1, 0.4))\
            .key_down(Keys.CONTROL)\
            .click(a)\
            .key_up(Keys.CONTROL)\
            .perform()
    except Exception:
        return False

    t_end = time.time() + 8
    while time.time() < t_end:
        if len(driver.window_handles) > len(handles_before):
            return True
        timer.wait(0.15, 0.35, extra_jitter=(0.0, 0.15))
    return False


def _leer_unidades_del_anuncio(driver: webdriver.Chrome, timer: HumanTimer) ->␣
 ↪Tuple[int, Optional[str], Optional[str], Optional[float], Optional[float]]:
    """
    RESTORE: devuelve SOLO:
      (unidades, ubicacion, maps_url, lat, lng)
    """
    try:
        WebDriverWait(driver, 12).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, "div.
 ↪ui-vip-location, #ui-vip-location__map img"))
        )
    except Exception:
        pass

    try:
        driver.execute_script("window.scrollTo(0, Math.min(1200, document.body.
 ↪scrollHeight));")
    except Exception:
        pass
    timer.wait(0.3, 0.9, extra_jitter=(0.0, 0.3))

    html = driver.page_source

    unidades = _contar_unidades_en_detalle_html(html)
    ubicacion_detalle, maps_url, lat, lng = _extraer_ubicacion_y_maps_html(html)

    # blindaje: Maps_url no debe ser staticmap nunca
    if maps_url and "maps.googleapis.com/maps/api/staticmap" in maps_url:
```

```
        maps_url = None

    return unidades, ubicacion_detalle, maps_url, lat, lng
```

[ ]:
```
# ================== SCRAPERUNNER - CELDA 6/7 (v2.28.1)
# - Extrae POI_Botones + 5 secciones:
#   Transporte, Educación, Áreas verdes, Comercios, Salud
# - Click tab-by-tab con "refresh" (click otro tab y regreso)
# - Guarda:
#   Transporte_POI, Escuelas_POI, AreasVerdes_POI, Comercios_POI, Salud_POI␣
 ↪(JSON)
#   Transporte_txt_all, Escuelas_txt_all, AreasVerdes_txt_all,␣
 ↪Comercios_txt_all, Salud_txt_all (texto crudo)
# - NO toca nada de Maps/Lat/Lng/CP ya existente (sigue usando␣
 ↪_leer_unidades_del_anuncio)

from typing import List, Dict, Tuple, Optional
import re
import json

def extraer_poi_botones(driver: webdriver.Chrome) -> str:
    try:
        container = driver.find_element(By.CSS_SELECTOR, "div.ui-vip-poi__tabs")
    except Exception:
        return ""
    botones = container.find_elements(By.CSS_SELECTOR, "button")
    titulos: List[str] = []
    for b in botones:
        try:
            t = b.find_element(By.CSS_SELECTOR, "span.ui-vip-poi__tab-title").
 ↪text.strip()
            if t:
                titulos.append(t)
        except Exception:
            continue
    return " | ".join(titulos)

# ---------- parsing mins/metros ----------
_RE_MINS = re.compile(r"(\d+(?:\.\d+)?)\s*mins?", re.IGNORECASE)
_RE_METROS = re.compile(r"([\d\.,]+)\s*metros", re.IGNORECASE)

def _parse_mins_metros(s: str) -> Tuple[Optional[float], Optional[float]]:
    if not s:
        return None, None
    mins = None
    metros = None
    m1 = _RE_MINS.search(s)
```

```
    if m1:
        try:
            mins = float(m1.group(1))
        except Exception:
            mins = None
    m2 = _RE_METROS.search(s)
    if m2:
        try:
            metros = float(m2.group(1).replace(",", "").replace(" ", ""))
        except Exception:
            metros = None
    return mins, metros

# ---------- clicks robustos ----------
def _click_tab(driver: webdriver.Chrome, timer: HumanTimer, btn) -> bool:
    try:
        driver.execute_script("arguments[0].scrollIntoView({block:'center'});",␣
 ↪btn)
    except Exception:
        pass
    timer.wait(0.15, 0.45, extra_jitter=(0.0, 0.15))
    try:
        ActionChains(driver).move_to_element(btn).pause(random.uniform(0.05, 0.
 ↪25)).click().perform()
        timer.wait(0.25, 0.65, extra_jitter=(0.0, 0.2))
        return True
    except Exception:
        try:
            driver.execute_script("arguments[0].click();", btn)
            timer.wait(0.25, 0.65, extra_jitter=(0.0, 0.2))
            return True
        except Exception:
            return False

def _esperar_panel(driver: webdriver.Chrome, panel_id: str, timeout_s: int = 8)␣
 ↪-> bool:
    try:
        WebDriverWait(driver, timeout_s).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, f"#{panel_id} .
 ↪ui-vip-poi__subsection, #{panel_id} .ui-vip-poi__item"))
        )
        return True
    except Exception:
        return False

def _extraer_items_de_panel(panel_el) -> Tuple[List[Dict], str]:
    """
```

```
    Devuelve:
      items: [{tipo, nombre, raw, mins, metros}, ...]
      txt_all: texto crudo consolidado (líneas)
    """
    out: List[Dict] = []
    lines: List[str] = []

    subsecs = panel_el.find_elements(By.CSS_SELECTOR, "div.
↪ui-vip-poi__subsection")
    for sub in subsecs:
        tipo = None
        try:
            tipo = sub.find_element(By.CSS_SELECTOR, "span.
↪ui-vip-poi__subsection-title > span").text.strip()
        except Exception:
            tipo = None

        items = sub.find_elements(By.CSS_SELECTOR, "div.
↪ui-vip-poi__item[data-testid='item']")
        for it in items:
            try:
                titulo = it.find_element(By.CSS_SELECTOR, "div.
↪ui-vip-poi__item-title span span").text.strip()
            except Exception:
                titulo = ""
            try:
                raw = it.find_element(By.CSS_SELECTOR, "div.
↪ui-vip-poi__item-subtitle span.ui-pdp-color--GRAY span").text.strip()
            except Exception:
                raw = ""

            mins, metros = _parse_mins_metros(raw)

            # nombre con prefijo tipo (si existe)
            if tipo and titulo:
                nombre = f"{tipo} {titulo}".strip()
            else:
                nombre = (titulo or "").strip()

            # línea cruda
            if tipo:
                line = f"{tipo} | {titulo} | {raw}".strip(" |")
            else:
                line = f"{titulo} | {raw}".strip(" |")
            if line:
                lines.append(line)
```

```
            out.append({
                "tipo": tipo,
                "nombre": nombre,
                "raw": raw,
                "mins": mins,
                "metros": metros
            })

    return out, "\n".join(lines)

def extraer_pois_5tabs(driver: webdriver.Chrome, timer: HumanTimer) ->␣
↪Dict[str, object]:
    """
    Extrae 5 tabs si existen:
      Transporte, Educación, Áreas verdes, Comercios, Salud

    Retorna dict con:
      POI_Botones
      Transporte_POI, Escuelas_POI, AreasVerdes_POI, Comercios_POI, Salud_POI
      Transporte_txt_all, Escuelas_txt_all, AreasVerdes_txt_all,␣
↪Comercios_txt_all, Salud_txt_all
    """
    res = {
        "POI_Botones": "",
        "Transporte_POI": [],
        "Escuelas_POI": [],
        "AreasVerdes_POI": [],
        "Comercios_POI": [],
        "Salud_POI": [],
        "Transporte_txt_all": "",
        "Escuelas_txt_all": "",
        "AreasVerdes_txt_all": "",
        "Comercios_txt_all": "",
        "Salud_txt_all": "",
    }

    # tabs container
    try:
        tabs = driver.find_element(By.CSS_SELECTOR, "div.ui-vip-poi__tabs")
    except Exception:
        return res

    buttons = tabs.find_elements(By.CSS_SELECTOR, "button[role='tab']")
    if not buttons:
        return res

    # map: titulo -> (button, panel_id)
```

```
    tab_map: Dict[str, Tuple[object, str]] = {}
    titles: List[str] = []

    for b in buttons:
        try:
            t = b.find_element(By.CSS_SELECTOR, "span.ui-vip-poi__tab-title").
↪text.strip()
        except Exception:
            continue
        if not t:
            continue
        panel_id = (b.get_attribute("aria-controls") or "").strip()
        if not panel_id:
            continue
        tab_map[t] = (b, panel_id)
        titles.append(t)

    res["POI_Botones"] = " | ".join(titles)

    # Orden deseado: primero Educación (porque a veces Transporte no carga si␣
↪es el default),
    # luego Transporte, y después las otras.
    # (Si no existe alguna, se salta.)
    desired = ["Educación", "Transporte", "Áreas verdes", "Comercios", "Salud"]

    # función para "refresh": click a otro tab y regreso al tab objetivo
    def click_with_refresh(target_title: str) -> Optional[object]:
        if target_title not in tab_map:
            return None
        target_btn, target_panel = tab_map[target_title]

        # elige "otro tab" cualquiera distinto
        other_btn = None
        for t in titles:
            if t != target_title and t in tab_map:
                other_btn = tab_map[t][0]
                break

        # refresh: click otro y regreso
        if other_btn is not None:
            _click_tab(driver, timer, other_btn)
            timer.wait(0.15, 0.45, extra_jitter=(0.0, 0.15))

        _click_tab(driver, timer, target_btn)
        _esperar_panel(driver, target_panel, timeout_s=8)
        try:
            panel_el = driver.find_element(By.ID, target_panel)
```

```
            return panel_el
        except Exception:
            return None

    # Extraer cada tab si existe
    for t in desired:
        panel_el = click_with_refresh(t)
        if panel_el is None:
            continue

        items, txt_all = _extraer_items_de_panel(panel_el)

        if t == "Transporte":
            res["Transporte_POI"] = items
            res["Transporte_txt_all"] = txt_all
        elif t == "Educación":
            # educación la manejas como "Escuelas_POI"
            res["Escuelas_POI"] = items
            res["Escuelas_txt_all"] = txt_all
        elif t == "Áreas verdes":
            res["AreasVerdes_POI"] = items
            res["AreasVerdes_txt_all"] = txt_all
        elif t == "Comercios":
            res["Comercios_POI"] = items
            res["Comercios_txt_all"] = txt_all
        elif t == "Salud":
            res["Salud_POI"] = items
            res["Salud_txt_all"] = txt_all

    return res


# ================== INTEGRACIÓN EN TU LOOP (NO TOCA LO DEMÁS)␣
 ↪==================

def scrapear_inmuebles_listado(driver: webdriver.Chrome) -> Tuple[pd.DataFrame,␣
 ↪pd.DataFrame, float]:
    timer = HumanTimer()
    data_raw: List[dict] = []
    data_limpio: List[dict] = []

    t0 = time.time()

    print(f"Abriendo: {BASE_URL}")
    driver.get(BASE_URL)
    timer.wait(2.5, 6.0, extra_jitter=(0.05, 0.8))
```

```
    page = PAGINA_INICIO

    while True:
        cards_bs4 = []
        soup = None

        for intento in range(2):
            human_scroll_page(driver, timer, min_steps=2, max_steps=5)
            html = driver.page_source
            soup_tmp = BeautifulSoup(html, "html.parser")
            cards_tmp = _extraer_cards_listado(soup_tmp)

            if cards_tmp:
                cards_bs4 = cards_tmp
                soup = soup_tmp
                break

            timer.wait(1.0, 3.0)

        if not cards_bs4:
            break

        cards_sel = driver.find_elements(By.CSS_SELECTOR, "div.
↪ui-search-result, div.ui-search-map-list__item")
        n = min(len(cards_bs4), len(cards_sel))
        if n == 0:
            break

        for i in range(n):
            card = cards_bs4[i]
            card_el = cards_sel[i]

            a_tag = card.find("a", href=True)
            if not a_tag:
                continue
            link = a_tag["href"]

            titulo_tag = card.find("h2")
            direccion = titulo_tag.get_text(strip=True) if titulo_tag else None

            price_tag = card.select_one(".andes-money-amount__fraction")
            if price_tag:
                precio = f"MXN {price_tag.get_text(strip=True)}"
            else:
                price_block = card.select_one(".ui-search-price")
                precio = price_block.get_text(" ", strip=True) if price_block␣
↪else None
```

```
            recamaras, superficie = extraer_recamaras_superficie(card)
            banos = extraer_banos_desde_texto(card.get_text(" ", strip=True))

            # ===== DETALLE =====
            unidades = 1
            ubicacion_detalle = None
            maps_url = None
            lat = None
            lng = None

            poi_botones = ""
            transporte_poi = []
            escuelas_poi = []
            areasverdes_poi = []
            comercios_poi = []
            salud_poi = []
            transporte_txt_all = ""
            escuelas_txt_all = ""
            areasverdes_txt_all = ""
            comercios_txt_all = ""
            salud_txt_all = ""

            listado_handle = driver.current_window_handle
            handles_before = driver.window_handles[:]

            opened = _abrir_anuncio_en_nueva_pestana(driver, timer, card_el)
            if opened:
                handles_after = driver.window_handles
                new_handles = [h for h in handles_after if h not in␣
↪handles_before]
                if new_handles:
                    driver.switch_to.window(new_handles[0])
                    timer.wait(0.6, 1.6)

                    try:
                        # LO QUE YA FUNCIONA (Maps_url blindado + Lat/Lng)
                        unidades, ubicacion_detalle, maps_url, lat, lng =␣
↪_leer_unidades_del_anuncio(driver, timer)

                        # NUEVO: POIs 5 tabs con clicks + refresh
                        poi = extraer_pois_5tabs(driver, timer)
                        poi_botones = poi.get("POI_Botones", "") or ""

                        transporte_poi = poi.get("Transporte_POI", []) or []
                        escuelas_poi = poi.get("Escuelas_POI", []) or []
                        areasverdes_poi = poi.get("AreasVerdes_POI", []) or []
```

```
                        comercios_poi = poi.get("Comercios_POI", []) or []
                        salud_poi = poi.get("Salud_POI", []) or []

                        transporte_txt_all = poi.get("Transporte_txt_all", "")␣
↪or ""
                        escuelas_txt_all = poi.get("Escuelas_txt_all", "") or ""
                        areasverdes_txt_all = poi.get("AreasVerdes_txt_all",␣
↪"") or ""
                        comercios_txt_all = poi.get("Comercios_txt_all", "") or␣
↪""
                        salud_txt_all = poi.get("Salud_txt_all", "") or ""

                    except Exception:
                        pass

                    try:
                        driver.close()
                    except Exception:
                        pass

                    driver.switch_to.window(listado_handle)
                    timer.wait(0.3, 0.8)

            # ===== RAW =====
            data_raw.append({
                "Link": link,
                "Direccion": direccion,
                "Ubicacion": ubicacion_detalle,
                "Maps_url": maps_url,
                "Lat": lat,
                "Lng": lng,

                "POI_Botones": poi_botones,

                # Texto crudo consolidado por sección
                "Transporte_txt_all": transporte_txt_all,
                "Escuelas_txt_all": escuelas_txt_all,
                "AreasVerdes_txt_all": areasverdes_txt_all,
                "Comercios_txt_all": comercios_txt_all,
                "Salud_txt_all": salud_txt_all,

                # JSON por sección
                "Transporte_POI": json.dumps(transporte_poi,␣
↪ensure_ascii=False),
                "Escuelas_POI": json.dumps(escuelas_poi, ensure_ascii=False),
                "AreasVerdes_POI": json.dumps(areasverdes_poi,␣
↪ensure_ascii=False),
```

```
                "Comercios_POI": json.dumps(comercios_poi, ensure_ascii=False),
                "Salud_POI": json.dumps(salud_poi, ensure_ascii=False),

                "Precio": precio,
                "Recamaras": recamaras,
                "Superficie_m2": superficie,
                "Banos": banos,
                "Unidades": unidades,
            })

            # ===== LIMPIO =====
            precio_num = parsear_precio_mxn(precio) if precio else np.nan
            if not np.isnan(precio_num):
                data_limpio.append({
                    "Link": link,
                    "Direccion": direccion,
                    "Ubicacion": ubicacion_detalle,
                    "Maps_url": maps_url,
                    "Lat": lat,
                    "Lng": lng,

                    "POI_Botones": poi_botones,

                    "Transporte_txt_all": transporte_txt_all,
                    "Escuelas_txt_all": escuelas_txt_all,
                    "AreasVerdes_txt_all": areasverdes_txt_all,
                    "Comercios_txt_all": comercios_txt_all,
                    "Salud_txt_all": salud_txt_all,

                    "Transporte_POI": json.dumps(transporte_poi,␣
↪ensure_ascii=False),
                    "Escuelas_POI": json.dumps(escuelas_poi,␣
↪ensure_ascii=False),
                    "AreasVerdes_POI": json.dumps(areasverdes_poi,␣
↪ensure_ascii=False),
                    "Comercios_POI": json.dumps(comercios_poi,␣
↪ensure_ascii=False),
                    "Salud_POI": json.dumps(salud_poi, ensure_ascii=False),

                    "Precio": precio,
                    "Precio_num": precio_num,
                    "Recamaras": recamaras,
                    "Superficie_m2": superficie,
                    "Banos": banos,
                    "Unidades": unidades,
                })
```

```
        if page >= PAGINA_FIN or not click_siguiente(driver, timer):
            break
        page += 1

    tiempo = time.time() - t0
    return pd.DataFrame(data_raw), pd.DataFrame(data_limpio), tiempo
```

[ ]:
```
# ================== SCRAPERUNNER - CELDA 7/7 (FIX v2.28.1)
# OBJETIVO:
# - NO tocar tu scraping que YA funciona (tabs + clicks + extracción de POIs + Maps/Lat/Lng/CP, etc.)
# - SOLO POST-PROCESO + GUARDADO:
#   1) Normaliza columnas
#   2) Expande JSONs de POIs a:
#        - <Prefijo>_txt_all (texto crudo concatenado)
#        - <Prefijo>_1_txt/_mins/_metros ... hasta el máximo global
#        - <Prefijo>_prom_mins / <Prefijo>_prom_metros
#   3) Guarda CSV (y XML). XLSX solo si existe openpyxl (si no, lo omite sin error)
#
# NOTA:
# - Esto elimina el error "No module named openpyxl" (ya no rompe).
# - Minimiza el PerformanceWarning usando concat en lugar de insertar columnas una por una.

import re
import json
import time
import unicodedata
from typing import Optional, Dict, Tuple, List

import numpy as np
import pandas as pd

# ================== Guardado ==================
def guardar_para_excel(df: pd.DataFrame, base_name: str):
    # CSV (siempre)
    try:
        df.to_csv(f"{base_name}.csv", index=False, encoding="utf-8-sig")
        print(f"Guardado {base_name}.csv")
    except Exception as e:
        print(f"No se pudo guardar {base_name}.csv:", e)

    # XLSX (solo si openpyxl existe)
    try:
        import openpyxl  # noqa: F401
        try:
```

```
            df.to_excel(f"{base_name}.xlsx", index=False)
            print(f"Guardado {base_name}.xlsx")
        except Exception as e:
            print(f"No se pudo guardar {base_name}.xlsx:", e)
    except Exception:
        # openpyxl no existe -> no hacemos nada (sin ruido)
        pass

    # XML (siempre, si se puede)
    try:
        df.to_xml(f"{base_name}.xml", index=False)
        print(f"Guardado {base_name}.xml")
    except Exception as e:
        print(f"No se pudo guardar {base_name}.xml:", e)

# ================== Normalización columnas ==================
def _normalize_colname(s: str) -> str:
    s = str(s).strip()
    s = unicodedata.normalize("NFKD", s)
    s = "".join(ch for ch in s if not unicodedata.combining(ch))
    s = re.sub(r"\s+", "_", s)
    return s

def normalizar_columnas(df: pd.DataFrame) -> pd.DataFrame:
    if df is None or df.empty:
        return df
    df.columns = [_normalize_colname(c) for c in df.columns]
    return df

# ================== CP desde texto (si ya lo usas; NO toca tu scraping)␣
 ↪==================
_CP_STRICT = re.compile(r"\b(?:c\.?\s*p\.?|cp|codigo\s+postal)\b[\s:.
 ↪\-]*([0-9]{5})\b", re.IGNORECASE)
_CP_FALLBACK = re.compile(r"\b([0-9]{5})\b")

def extraer_cp_desde_texto(ubicacion: Optional[str]) -> Optional[str]:
    if ubicacion is None:
        return None
    if isinstance(ubicacion, float) and np.isnan(ubicacion):
        return None
    txt = str(ubicacion)

    m = _CP_STRICT.search(txt)
    if m:
        return m.group(1)

    m2 = _CP_FALLBACK.search(txt)
```

```
    if m2:
        return m2.group(1)

    return None

# ================== (Opcional) CP desde coords (si lo sigues usando)␣
 ↪==================
# Si ya NO quieres Nominatim, pon USAR_NOMINATIM = False
USAR_NOMINATIM = True

from urllib.parse import urlencode
from urllib.request import Request, urlopen

def _nominatim_postcode_urllib(lat: float, lng: float, timeout=20) ->␣
 ↪Optional[str]:
    base = "https://nominatim.openstreetmap.org/reverse"
    params = {"format": "jsonv2", "lat": lat, "lon": lng, "zoom": 18,␣
 ↪"addressdetails": 1}
    url = base + "?" + urlencode(params)
    req = Request(url, headers={"User-Agent": "WebScraping2.28 (postcode lookup;
 ↪ non-commercial)"})
    try:
        with urlopen(req, timeout=timeout) as resp:
            raw = resp.read().decode("utf-8", errors="ignore")
        js = json.loads(raw)
        addr = js.get("address", {}) if isinstance(js, dict) else {}
        cp = addr.get("postcode")
        if not cp:
            return None
        m = re.search(r"\b([0-9]{5})\b", str(cp))
        return m.group(1) if m else str(cp).strip()
    except Exception:
        return None

def llenar_cp_con_nominatim(df: pd.DataFrame, col_lat="Lat", col_lng="Lng",␣
 ↪col_cp="CP",
                            max_requests: int = 300, sleep_s: float = 1.05) ->␣
 ↪pd.DataFrame:
    if df is None or df.empty:
        return df

    if col_cp not in df.columns:
        df[col_cp] = None

    mask = df[col_cp].isna() & df[col_lat].notna() & df[col_lng].notna()
    idxs = df.index[mask].tolist()
```

```
    if not idxs:
        return df

    cache: Dict[Tuple[float, float], Optional[str]] = {}
    reqs = 0
    hits = 0

    for idx in idxs:
        if reqs >= max_requests:
            break
        try:
            lat = float(df.at[idx, col_lat])
            lng = float(df.at[idx, col_lng])
        except Exception:
            continue

        key = (round(lat, 5), round(lng, 5))
        if key in cache:
            df.at[idx, col_cp] = cache[key]
            hits += 1
            continue

        cp = _nominatim_postcode_urllib(lat, lng)
        cache[key] = cp
        df.at[idx, col_cp] = cp
        reqs += 1
        time.sleep(sleep_s)

    print(f"[CP] Nominatim: requests={reqs} cache_hits={hits}␣
 ↪max={max_requests}")
    return df

# ================== EXPANDIR POIs JSON -> COLUMNAS DINÁMICAS (5 secciones)␣
 ↪==================
FALTA_DATO_TXT = "No hay más datos disponibles"

def _safe_json_load(x):
    if x is None:
        return []
    if isinstance(x, float) and np.isnan(x):
        return []
    if isinstance(x, list):
        return x
    s = str(x).strip()
    if not s:
        return []
    try:
```

```
        return json.loads(s)
    except Exception:
        return []

def _to_float(x):
    try:
        if x is None:
            return np.nan
        if isinstance(x, (int, float)):
            return float(x)
        s = str(x).strip().replace(",", "")
        return float(s)
    except Exception:
        return np.nan

def _mean_ignore_nan(vals: List[float]) -> float:
    clean = []
    for v in vals:
        try:
            if v is None:
                continue
            if isinstance(v, float) and np.isnan(v):
                continue
            clean.append(float(v))
        except Exception:
            continue
    return float(np.mean(clean)) if clean else np.nan

def _build_txt_all(items: list) -> str:
    # Cada item esperado: {"tipo":..., "nombre":..., "raw":..., "mins":...,␣
 ↪"metros":...}
    lines = []
    for it in items or []:
        nombre = (it.get("nombre") or "").strip()
        raw = (it.get("raw") or "").strip()
        if nombre and raw:
            lines.append(f"{nombre} | {raw}")
        elif nombre:
            lines.append(nombre)
        elif raw:
            lines.append(raw)
    return " || ".join(lines)

def expandir_pois_5_secciones(df: pd.DataFrame) -> pd.DataFrame:
    """
    Espera (si existen) columnas JSON:
      - Transporte_POI
```

```
    - Escuelas_POI
    - AreasVerdes_POI
    - Comercios_POI
    - Salud_POI

  Genera para cada sección (si existe):
    - <Seccion>_txt_all
    - <Seccion>_1_txt/_mins/_metros ... hasta el máximo global de esa sección
    - <Seccion>_prom_mins / <Seccion>_prom_metros
  """
  if df is None or df.empty:
      return df

  secciones = [
      ("Transporte",   "Transporte_POI"),
      ("Escuelas",     "Escuelas_POI"),
      ("AreasVerdes",  "AreasVerdes_POI"),
      ("Comercios",    "Comercios_POI"),
      ("Salud",        "Salud_POI"),
  ]

  existentes = [(pref, col) for (pref, col) in secciones if col in df.columns]
  if not existentes:
      return df

  # Parsear JSONs una vez
  parsed = {}
  max_len = {}
  for pref, col in existentes:
      s = df[col].apply(_safe_json_load)
      parsed[pref] = s
      max_len[pref] = int(s.apply(len).max() or 0)

  # Preparar columnas nuevas en bloque (evita fragmentation)
  newcols = {}

  for pref, _col in existentes:
      m = max_len[pref]

      # txt_all
      newcols[f"{pref}_txt_all"] = parsed[pref].apply(_build_txt_all)

      # items 1..m
      for k in range(1, m + 1):
          newcols[f"{pref}_{k}_txt"] = FALTA_DATO_TXT
          newcols[f"{pref}_{k}_mins"] = np.nan
          newcols[f"{pref}_{k}_metros"] = np.nan
```

```
    newdf = pd.DataFrame(newcols, index=df.index)

    # Llenar items
    for pref, _col in existentes:
        m = max_len[pref]
        if m <= 0:
            # promedios vacíos
            newdf[f"{pref}_prom_mins"] = np.nan
            newdf[f"{pref}_prom_metros"] = np.nan
            continue

        for idx in df.index:
            items = parsed[pref].at[idx] if idx in parsed[pref].index else []
            # llenar hasta m; el resto ya queda con "No hay más..."
            for j, it in enumerate(items, start=1):
                if j > m:
                    break
                nombre = (it.get("nombre") or "").strip()
                raw = (it.get("raw") or "").strip()
                mins = _to_float(it.get("mins"))
                metros = _to_float(it.get("metros"))

                txt = f"{nombre} | {raw}".strip(" |")
                if not txt:
                    txt = FALTA_DATO_TXT

                newdf.at[idx, f"{pref}_{j}_txt"] = txt
                newdf.at[idx, f"{pref}_{j}_mins"] = mins
                newdf.at[idx, f"{pref}_{j}_metros"] = metros

        # promedios
        mins_cols = [f"{pref}_{k}_mins" for k in range(1, m + 1)]
        met_cols  = [f"{pref}_{k}_metros" for k in range(1, m + 1)]

        newdf[f"{pref}_prom_mins"] = newdf[mins_cols].apply(lambda r:␣
 ↪_mean_ignore_nan(r.values.tolist()), axis=1)
        newdf[f"{pref}_prom_metros"] = newdf[met_cols].apply(lambda r:␣
 ↪_mean_ignore_nan(r.values.tolist()), axis=1)

    # Unir todo de golpe
    df_out = pd.concat([df, newdf], axis=1)
    return df_out

# ================== MAIN (solo orquesta; NO cambia tu scraping)␣
 ↪==================
def main():
```

```
    driver = crear_driver_persistente()
    try:
        esperar_login_y_volver(driver, BASE_URL)
        df_raw, df_limpio, tiempo = scrapear_inmuebles_listado(driver)
    finally:
        driver.quit()

    if df_raw is None or df_raw.empty:
        print("No se obtuvieron datos. No se guarda.")
        return {"ok": False, "raw": 0, "limpio": 0, "tiempo": tiempo}

    # normalizar columnas
    df_raw = normalizar_columnas(df_raw)
    df_limpio = normalizar_columnas(df_limpio)

    # limpieza compat
    for df in (df_raw, df_limpio):
        if "Ubicacion_card" in df.columns:
            df.drop(columns=["Ubicacion_card"], inplace=True)

    # CP por texto (si no existe CP aún)
    if "CP" not in df_raw.columns:
        df_raw["CP"] = None
    if "Ubicacion" in df_raw.columns:
        mask_cp = df_raw["CP"].isna()
        df_raw.loc[mask_cp, "CP"] = df_raw.loc[mask_cp, "Ubicacion"].
↪apply(extraer_cp_desde_texto)

    # fallback coords (si lo quieres)
    if USAR_NOMINATIM:
        df_raw = llenar_cp_con_nominatim(df_raw, col_lat="Lat", col_lng="Lng",␣
↪col_cp="CP",
                                         max_requests=300, sleep_s=1.05)

    # propagar CP a limpio
    if df_limpio is not None and not df_limpio.empty and "Link" in df_raw.
↪columns and "Link" in df_limpio.columns:
        cp_map = df_raw[["Link", "CP"]].dropna().
↪drop_duplicates(subset=["Link"])
        df_limpio = df_limpio.drop(columns=["CP"], errors="ignore").
↪merge(cp_map, on="Link", how="left")

    # EXPANDIR POIs (5 secciones)
    df_raw = expandir_pois_5_secciones(df_raw)
    df_limpio = expandir_pois_5_secciones(df_limpio)

    guardar_para_excel(df_raw, RAW_BASENAME)
```

```
    guardar_para_excel(df_limpio, LIMPIO_BASENAME)

    return {"ok": True, "raw": len(df_raw), "limpio": len(df_limpio), "tiempo":␣
 ↪tiempo}

# ================== EXPORT MAIN (para runner) ==================
try:
    __SCRAPER_MAIN__ = main
except NameError:
    __SCRAPER_MAIN__ = None
```

[ ]:
```
# ================== EXPANDIR POI JSON -> COLUMNAS DINÁMICAS (v2.28.3)
FALTA_DATO_TXT = "No hay más datos disponibles"

def _safe_json_load(x):
    if x is None:
        return []
    if isinstance(x, float) and np.isnan(x):
        return []
    if isinstance(x, list):
        return x
    s = str(x).strip()
    if not s:
        return []
    try:
        obj = json.loads(s)
        return obj if isinstance(obj, list) else []
    except Exception:
        return []

def _to_float(x):
    try:
        if x is None:
            return np.nan
        if isinstance(x, (int, float)):
            return float(x)
        s = str(x).strip().replace(",", "")
        return float(s)
    except Exception:
        return np.nan

def _mean_ignore_nan(arr):
    vals = [v for v in arr if isinstance(v, (int, float)) and not␣
 ↪(isinstance(v, float) and np.isnan(v))]
    return float(np.mean(vals)) if vals else np.nan
```

```
def _expand_one(df: pd.DataFrame, col_json: str, prefix_txt: str, prom_prefix:␣
 ↪str) -> pd.DataFrame:
    if col_json not in df.columns:
        return df

    lists = df[col_json].apply(_safe_json_load)
    max_n = int(lists.apply(len).max() or 0)

    # crear columnas
    for k in range(1, max_n + 1):
        df[f"{prefix_txt}_{k}_txt"] = FALTA_DATO_TXT
        df[f"{prefix_txt}_{k}_mins"] = np.nan
        df[f"{prefix_txt}_{k}_metros"] = np.nan

    # llenar
    for idx in df.index:
        items = lists.at[idx] if idx in lists.index else []
        for j, it in enumerate(items, start=1):
            if j > max_n:
                break
            it = it if isinstance(it, dict) else {}
            nombre = (it.get("nombre") or "").strip()
            raw = (it.get("raw") or "").strip()
            mins = _to_float(it.get("mins"))
            metros = _to_float(it.get("metros"))

            txt = f"{nombre} | {raw}".strip(" |")
            if not txt:
                txt = FALTA_DATO_TXT

            df.at[idx, f"{prefix_txt}_{j}_txt"] = txt
            df.at[idx, f"{prefix_txt}_{j}_mins"] = mins
            df.at[idx, f"{prefix_txt}_{j}_metros"] = metros

    # promedios
    if max_n > 0:
        mins_cols = [f"{prefix_txt}_{k}_mins" for k in range(1, max_n + 1)]
        met_cols  = [f"{prefix_txt}_{k}_metros" for k in range(1, max_n + 1)]
        df[f"{prom_prefix}_prom_mins"] = df[mins_cols].apply(lambda r:␣
 ↪_mean_ignore_nan(r.values.tolist()), axis=1)
        df[f"{prom_prefix}_prom_metros"] = df[met_cols].apply(lambda r:␣
 ↪_mean_ignore_nan(r.values.tolist()), axis=1)
    else:
        df[f"{prom_prefix}_prom_mins"] = np.nan
        df[f"{prom_prefix}_prom_metros"] = np.nan

    return df
```

```python
def expandir_poi_json_a_columnas(df: pd.DataFrame) -> pd.DataFrame:
    """
    ENTRADA (JSON):
      - Transporte_POI, Escuelas_POI, AreasVerdes_POI, Comercios_POI, Salud_POI
    SALIDA:
      - Transporte_1_txt/mins/metros ... + Transporte_prom_*
      - Escuela_1_txt/mins/metros ...    + Escuelas_prom_*
      - AreasVerdes_1_* ...              + AreasVerdes_prom_*
      - Comercios_1_* ...                + Comercios_prom_*
      - Salud_1_* ...                    + Salud_prom_*
    """
    if df is None or df.empty:
        return df

    df = _expand_one(df, "Transporte_POI",   "Transporte",   "Transporte")
    df = _expand_one(df, "Escuelas_POI",     "Escuela",      "Escuelas")
    df = _expand_one(df, "AreasVerdes_POI",  "AreasVerdes",  "AreasVerdes")
    df = _expand_one(df, "Comercios_POI",    "Comercios",    "Comercios")
    df = _expand_one(df, "Salud_POI",        "Salud",        "Salud")

    return df
```